PID-RS MATHEMATICAL AUDIT

Source-marginal factorization and a bounded exact audit of a declared categorical SxPID3 transcription

A self-contained account of source-marginal factorization, the 18/108/166 distinction, exact finite-count products, bounded two-route agreement, negative results, and the remaining proof and refinement obligations.

3 September 2026

Canonical narrative: `SXPID3_SOURCE_MARGINAL_AND_BOUNDED_AUDIT.md`

Contents

Abstract

This report documents three related but logically separate project results for a supplied local transcription intended to represent the finite categorical shared-exclusions PID of Makkeh, Gutknecht, and Wibral (MGW). First, a generic finite-alphabet theorem shows that every averaged informative cumulative factors exactly through the complete joint source marginal. Consequently, any one fixed linear transform of those cumulatives—including a separately justified Möbius inverse—is constant when that complete source marginal is held fixed, even if the finite target alphabet or target-conditioned allocation changes. A counterexample proves that this invariance does not transfer to the misinformative or signed-net components.

Second, a bounded executable audit evaluates 108 keyed scalar expressions on every labelled binary 16-cell count vector with total count $1 \le N \le 5$. Here $108 = 18 \times 2 \times 3$: 18 three-source antichain positions, two representation stages, and three components. The number 166 instead belongs to the four-source redundancy lattice. Two implementation-disjoint-under-shared-semantics exact routes produced the same route-neutral digest and the same exact six-block sign/zero census under a source-bound execution receipt.

Third, an owner-controlled revision-5 source review now binds the relevant MGW equations to exact PDF, source-archive, and TeX-member identities; a fresh acquisition replay reproduces those identities, and ten anchor records make every semantic transfer boundary explicit. An executable semantic bridge reconstructs the three-source OR-of-AND event kernel, all 18 antichains, all 324 order/zeta entries, the exact two-sided Möbius inverse, and all six source-label permutations from small definitions, then compares the carrier and lattice exactly with the frozen conventions and prior route registries. Minimal counterexamples distinguish the central connective, target-intersection, order, weighting, and relabeling errors. This bridge reduces the shared-semantics risk beneath the two arithmetic routes; it does not eliminate the need for independent source review or concrete formal and Rust refinement.

Neither result defines a new PID. Williams--Beer $I_{\min}$, BROJA, the Ehrlich continuous shared-exclusions estimator, and KSG are not imported or identified with the categorical quantity studied here. The bounded audit is exhaustive only on its declared sparse binary count domain. It does not establish independently reviewed paper-to-code correspondence, compiled Rust numerical refinement, population validity, estimator calibration, causality, or scientific priority.

Status and claim boundary

This report explains three separate project-defined assurance results around a supplied transcription of the paper-defined categorical shared-exclusions PID:

1. **Informative source-marginal factorization.** Under the finite-alphabet assumptions stated in Section 1, an averaged informative cumulative depends on the complete joint source marginal and not on the target-conditioned allocation. Equality survives any one fixed finite linear transform.
2. **Bounded SxPID3 keyed-scalar agreement.** Under the exact binary-count domain in Section 4, two exact routes agree on a declared stream digest and complete sign/zero census for 108 keyed scalar audit expressions.
3. **Owner-controlled MGW semantic bridge.** Exact source identities and equation anchors are recorded, then the finite three-source carrier, event truth table, order, zeta matrix, Möbius inverse, and source-label automorphisms are regenerated from definitions. The bridge is a strong partial Program A result; it is not an independent source review or an end-to-end certificate.

These results do **not** correct the MGW/Wibral definition, machine-interpret the publication, complete independent source correspondence, certify compiled Rust numerics, or supply a statistical or application theorem. The method remains paper-defined; the source map, factorization proof, formalization, exact routes, hostile tests, receipt lifecycle, and this report are project-defined assurance.

The historical labels “P1” and “P5” are avoided because the claim packet used similar labels for other obligations. This report uses **factorization result** and **bounded audit**.

The current claim-status pointer is the evidence-adjudication index, which identifies decision record 3 as current. That decision accepts the three scoped results above without changing claim revision 1: the prospective complete certificate remains proposed/open, and Programs A--E closed remains zero of five. The packet's older `revision-index.md` is retained as byte-pinned historical intake, not used as the current decision pointer.

Provenance firewall

Ingredient	Provenance	Exact role here	Not transferred
Informative, misinformative, and signed-net categorical shared exclusions	MGW (2021)	Functional the audit is intended to transcribe	No automatic paper-to-checker correspondence
Antichain carrier and redundancy order	Williams and Beer (2010)	Lattice carrier and order	The $I_{\min}$ redundancy value is not used
Finite-poset Möbius inversion	Rota (1964)	Passage from cumulatives to atoms	No PID-specific conclusion follows from inversion alone
Revision-5 source identities and semantic anchors	MGW (2021), owner-controlled repository review	Binds the intended event, split, averaging, order, and Möbius statements to exact source locations	No authenticity, independent-review, parser, formal, or Rust credit
Source-marginal factorization	Project analysis of the paper-defined informative component	Exact theorem, fixed-transform corollary, and prohibited-transfer boundary	No new-PID or priority claim
Registry, finite corpus, census, neutral stream, and receipt	Project-defined validation	Bounded software assurance for a declared transcription	No general theorem, estimator, or population claim

Sharing an antichain lattice does not identify two PID measures. Every PID-specific conclusion below concerns categorical MGW shared exclusions only.

1. Objects and notation

Standing assumptions for this section. Fix nonempty finite source alphabets $\mathcal{S}_1, \mathcal{S}_2, \mathcal{S}_3$, a nonempty finite target alphabet $\mathcal{T}$, and one probability law P on their complete Cartesian product, including zero-mass cells. All logarithms are natural, so information is measured in nats. MGW write these definitions with $\log_2$ in bits; for the same positive probability ratio $r > 0$, the repository value is $\log r = (\log 2) \log_2 r$. This fixed positive change of units preserves every equality and sign used below but changes numerical magnitudes. Define

$$\mathcal{S} = \mathcal{S}_1 \times \mathcal{S}_2 \times \mathcal{S}_3, \qquad \mathcal{Z} = \mathcal{S} \times \mathcal{T}.$$

Let $S = (S_1, S_2, S_3)$ and T be the coordinate projections. The complete joint source marginal is

$$P_S(s) = \sum_{t \in \mathcal{T}} P(s, t).$$

"Source marginal" always means the distribution of the complete tuple (S_1, S_2, S_3). It does not mean the three separate marginals and does not assume source independence.

A three-source redundancy-lattice position α is a nonempty antichain of nonempty subsets of $\{1, 2, 3\}$. For $s \in \mathcal{S}$, define the source-only shared-exclusions event

$$E_\alpha(s) = \{s' \in \mathcal{S} : \exists a \in \alpha \; \forall i \in a, \; s'_i = s_i\}\,.$$

This is OR across antichain branches and AND within one branch. It is the local event supplied by the repository transcription, with MGW (2021), Equation (6), as its paper anchor. The owner-controlled source map and finite reconstruction in Sections 1.1 and 7.3 support that mapping; independent source review and concrete formal correspondence remain separate obligations. In the complete source-target space the supplied event is the cylinder

$$E_\alpha(s) \times \mathcal{T}.$$

The target coordinate is absent from event membership. That typing fact is more informative than saying that a target term happens to cancel numerically.

1.1 Paper event semantics: Equation (4) is not Equation (6)

The source correspondence uses MGW revision 5, arXiv `2002.03356v5`. The exact SHA-256 values are shown in four blocks; concatenating each row without the displayed separators gives the 64-digit digest:

- PDF: `5939ce0f4c727f19 · 98040421c07a1689 · af1b8d9a35a0ee3c · 83fe25cd85263dc6`;
- source archive: `6420b90ccd5c1e97 · 1e19b41c24676b0e · d3276aa47f1b3cea · dc49bac219bf9584`;
- extracted `apstemplate.tex`: `60ac061c9874149d · 65d6fab21e627ca6 · 6f96d9e4d4990d1e · e243632776faaf61`.

Their respective byte counts are 1,002,114, 489,040, and 142,869. A fresh owner-controlled HTTPS replay ran on 2026-09-03 UTC. It reacquired both external artifacts and verified that the archive contained exactly one top-level `apstemplate.tex`, and reproduced every size and digest. The exact replay commands, results, source-line intervals, semantic roles, and prohibited inferences are retained in the source-correspondence record. These hashes identify the reviewed bytes. They do not authenticate the publisher or interpret the mathematics automatically; this replay is not independent custody or trusted time.

MGW Equation (4) and Equations (5)--(8) use related notation for different objects:

- Equation (4) begins with a subset of *individual source realizations*. The event is an OR among the selected individual equality statements. In this report's mask notation, choosing sources 1 and 2 separately gives antichain key `01+02`.
- Equation (6) permits a collection of source collections. Inside one collection, all named source equalities are joined by AND; different collections are joined by OR. The joint collection of sources 1 and 2 is key `03`.

Thus, for a keyed value (s_1, s_2, s_3),

$$E_{01+02} = \{S_1 = s_1\} \cup \{S_2 = s_2\},$$

whereas

$$E_{03} = \{S_1 = s_1\} \cap \{S_2 = s_2\}.$$

On the equality pattern `010` - source 2 matches, while sources 1 and 3 do not - `01+02` is true and `03` is false. This one-row witness rejects both AND across antichain branches and OR inside a joint-source collection. It prevents a notation-based substitution of Equation (4) for Equation (6).

The other load-bearing source anchors are:

MGW location	Local role	Boundary that remains
Equation (12)	Equivalent exclusion-probability form	Population equivalence is not Rust refinement
Equation (13)	Cumulative is the zeta sum of lower-or-equal atoms	The matrix orientation must still be stated
Equations (14a), (15a), (15b)	Net equals informative minus misinformative	Component signs do not transfer to net atoms
Equation (17)	Average local values with the complete joint source-target law	The weight is not event probability or uniform key weight
Appendix A order display	Quantified antichain order	Reversing the order is a different convention
Appendix Equation (A1), Theorem A.1	Componentwise Möbius inversion	Matrix inversion alone does not verify event semantics
Theorem IV.2	Informative and misinformative cumulatives increase on the redundancy lattice	No inference of atom nonnegativity or signed-net monotonicity
Theorem IV.3	Full-lattice pointwise component-atom nonnegativity	No transfer to net atoms, truncations, or other PIDs

This is an owner-controlled source review. The detailed map records exact locations and explicitly separates the paper meaning, repository analogue, preserved assumptions, changed conventions, and prohibited inference. It also records the evidence required before each anchor can support a stronger formal or implementation claim. In compact form:

- **Equations (4) and (5)--(8).** Finite categorical equality-event semantics at one realized source tuple is preserved. Source collections become sorted masks and stable keys. Independent source review, formal event semantics, and parser-to-Rust refinement remain required.
- **Equation (12).** The source event, target event, and positive log-ratio probabilities are preserved. The exclusion form is used only as a cross-check. Algebraic correspondence and a numerical or compiled comparison are required before that form is used operationally.
- **Equation (13), the Appendix order, and Equation (A1).** The complete carrier, published order, and componentwise finite-poset inversion are preserved. The repository declares row/column orientation, stable key order, and exact rational elimination. Concrete carrier/order formalization and implementation correspondence remain required.
- **Equations (14a), (15a), and (15b).** The paper-defined pointwise informative/misinformative split is preserved. Bits become nats by multiplication with positive $\ln 2$. Componentwise correspondence remains required, and signed net needs a separate subtraction analysis.
- **Equation (17).** The complete joint source-target expectation over positive-mass keys is preserved. Probabilities become c_z/N weights for the declared plug-in law. Producer/parser refinement must establish the same weighting.
- **Theorems IV.2 and IV.3.** The full lattice and separate informative/misinformative components are preserved. Only notation and the positive unit scale change. Source-to-formal correspondence remains required; neither theorem transfers to signed net, a truncated carrier, or another PID.

The fresh acquisition remains owner-controlled. Independent acquisition, review, and external custody remain open.

1.2 The 18/108/166 crosswalk

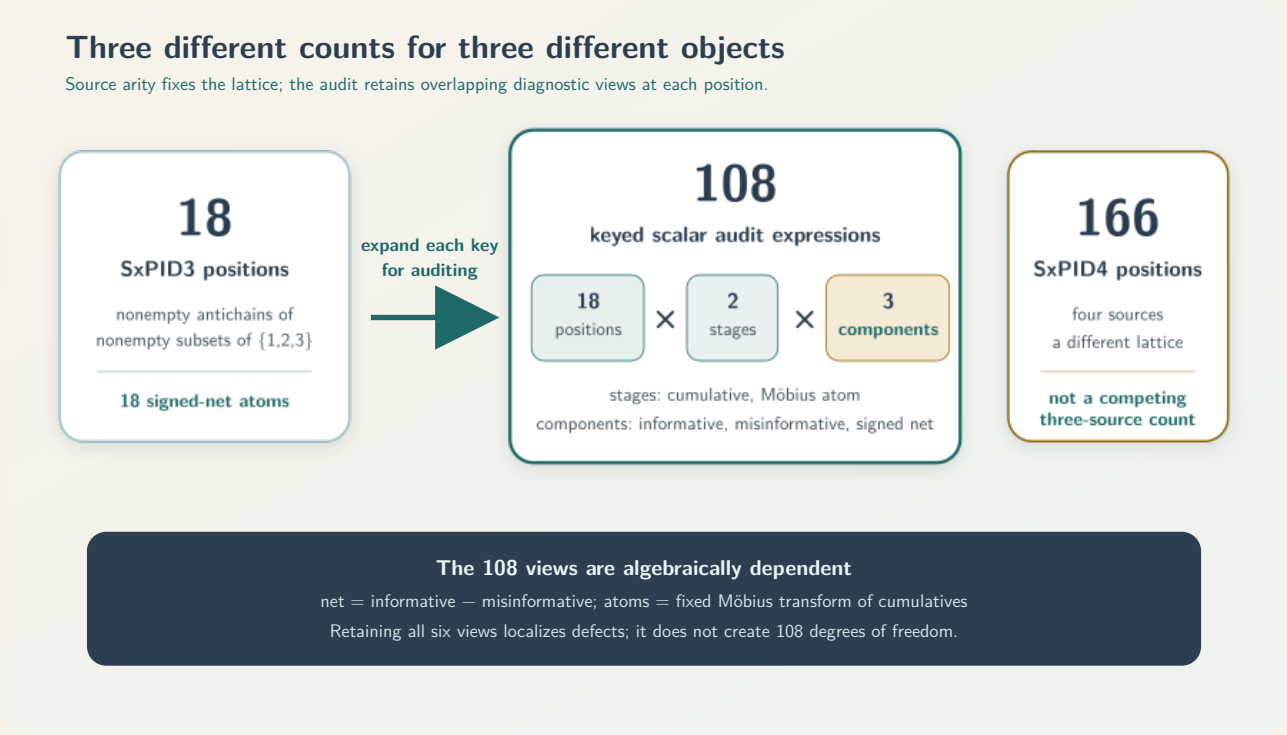
The source arity determines the carrier. Let $\mathcal{L}_n$ be the nonempty antichains of nonempty subsets of $\{1, \dots, n\}$. The Dedekind number $M(n)$ counts all antichains of the full Boolean lattice. Removing the empty antichain and the singleton antichain $\{\emptyset\}$ gives, under this declared carrier convention,

$$|\mathcal{L}_3| = M(3) - 2 = 20 - 2 = 18,$$

and

$$|\mathcal{L}_4| = M(4) - 2 = 168 - 2 = 166.$$

Object	Count
SxPID3 antichain positions	18
Usual SxPID3 signed-net atoms	18
One three-component cumulative block	$18 \times 3 = 54$
One three-component atom block	$18 \times 3 = 54$
Complete audit vector	$54 + 54 = 108$
SxPID4 antichain positions and signed-net atoms	166



The exact audit registry is

$$\mathcal{R} = \mathcal{L}_3 \times \{\text{cumulative}, \text{atom}\} \times \{+, -, \text{net}\},$$
$$|\mathcal{R}| = 18 \cdot 2 \cdot 3 = 108.$$

The 108 entries are not independent. Under the component convention used here,

$$C^{\text{net}} = C^{+} - C^{-},$$

and, for one fixed Möbius inverse M,

$$\Pi^u = MC^u, \qquad \Pi^{\mathrm{net}} = \Pi^+ - \Pi^-.$$

Once C^+ and C^- are known, the other four blocks are determined. Retaining all six blocks is still valuable: it localizes event, component, lattice-orientation, inversion, and subtraction defects that a final signed value could hide through cancellation.

1.3 Redundancy order, zeta transform, and the fixed 18-position registry

Standing assumptions for this subsection. Use the three-source antichain carrier just defined, represent nonempty source subsets by masks `01` through `07`, and sort a multi-mask key first by its number of masks (the antichain cardinality) and then lexicographically.

Index	Stable key	Source collections
0	`01`	$\{\{1\}\}$
1	`02`	$\{\{2\}\}$
2	`03`	$\{\{1,2\}\}$
3	`04`	$\{\{3\}\}$
4	`05`	$\{\{1,3\}\}$
5	`06`	$\{\{2,3\}\}$
6	`07`	$\{\{1,2,3\}\}$
7	`01+02`	$\{\{1\},\{2\}\}$
8	`01+04`	$\{\{1\},\{3\}\}$
9	`01+06`	$\{\{1\},\{2,3\}\}$
10	`02+04`	$\{\{2\},\{3\}\}$
11	`02+05`	$\{\{2\},\{1,3\}\}$
12	`03+04`	$\{\{1,2\},\{3\}\}$
13	`03+05`	$\{\{1,2\},\{1,3\}\}$
14	`03+06`	$\{\{1,2\},\{2,3\}\}$
15	`05+06`	$\{\{1,3\},\{2,3\}\}$
16	`01+02+04`	$\{\{1\},\{2\},\{3\}\}$
17	`03+05+06`	$\{\{1,2\},\{1,3\},\{2,3\}\}$

Define the redundancy order by

$$\alpha \preceq \beta \quad \Longleftrightarrow \quad \forall b \in \beta\, \exists a \in \alpha :\ a \subseteq b.$$

MGW Equation (13) supplies the pointwise zeta relation for the signed-net component; Equations (14a), (15a), and (15b) define the split, and their Appendix A supplies componentwise Möbius inversion. For every supported z, and for $u \in \{+,-,\mathrm{net}\}$, the pointwise relation is $i_i^u(z) = \sum_j Z_{ij}\pi_j^u(z)$. Equation (17), applied componentwise, then commutes with that finite zeta sum:

$$\begin{aligned} C_i^u &= \sum_{z:P(z)>0} P(z) i_i^u(z) \\ &= \sum_{z:P(z)>0} P(z) \sum_j Z_{ij}\pi_j^u(z) \\ &= \sum_j Z_{ij} \sum_{z:P(z)>0} P(z)\pi_j^u(z). \end{aligned}$$

Writing $\Pi_j^u = \sum_{z:P(z)>0} P(z)\pi_j^u(z)$ gives the averaged zeta relation below. The support restriction avoids the undefined expression 0 times a local value outside its domain. This short bridge is why

pointwise inversion may be reused after averaging; it does not by itself establish that the supplied registry and order are the paper's intended ones.

With cumulatives as rows and atoms as columns, define

$$Z_{ij} = \mathbf{1}\{\alpha_j \preceq \alpha_i\}, \qquad C_i^u = \sum_{j=0}^{17} Z_{ij}\Pi_j^u, \qquad M = Z^{-1}.$$

Thus

$$\Pi_i^u = \sum_{j=0}^{17} M_{ij} C_j^u.$$

The complete row signatures and sparse inverse are frozen in `claims/SX-CERTIFIED-AVERAGED-PID3-001/conventions.md`. The matrix has 129 ones; its inverse has 65 nonzero entries in $\{-1, 1\}$; and both $MZ = I_{18}$ and $ZM = I_{18}$ are required. Those identities alone do not prove that the entries implement the paper-defined event semantics.

1.4 A two-source inversion example

Standing assumptions for this example. Use the conventional two-source four-position carrier, the same down-set orientation as above, and one component u. The label 1.2 denotes the antichain $\{\{1\},\{2\}\}$; it is not a decimal number. Writing $C_1, C_2, C_{12}, C_{1.2}$ for the corresponding cumulatives, Möbius inversion gives atoms such as

$$\Pi_1 = C_1 - C_{1.2}, \qquad \Pi_2 = C_2 - C_{1.2}.$$

The example shows why the direction of Z and the distinction between cumulative and atom stages are load-bearing. It is illustrative only; it is not evidence for the 18-position transcription.

2. Informative source-marginal factorization

Standing assumptions for this section. Continue with the probability law P on the finite product $\mathcal{S} \times \mathcal{T}$ fixed in Section 1. Evaluate local values only at supported (s,t), so $P(s,t) > 0$. Define the target fibre $F_t = \mathcal{S} \times \{t\}$. The anchored event contains s, so $P(E_\alpha(s) \times \mathcal{T})$, $P((E_\alpha(s) \times \mathcal{T}) \cap F_t)$, and $P(F_t)$ are all positive and every logarithm below is defined.

For a supported realization, the local informative, misinformative, and signed-net components are

$$\begin{aligned} i_\alpha^+(s,t;P) &= -\log P(E_\alpha(s) \times \mathcal{T}), \\ i_\alpha^-(s,t;P) &= \log \frac{P(F_t)}{P((E_\alpha(s) \times \mathcal{T}) \cap F_t)}, \\ i_\alpha^{\mathrm{net}}(s,t;P) &= i_\alpha^+(s,t;P) - i_\alpha^-(s,t;P) \\ &= \log \frac{P((E_\alpha(s) \times \mathcal{T}) \cap F_t)}{P(E_\alpha(s) \times \mathcal{T}) P(F_t)}. \end{aligned}$$

The source anchors used by this transcription are MGW Equations (14a), (15a), and (15b) for the directed local split and Equation (17) for the joint-law averaging convention, applied to the split componentwise by linearity. Citing those equations does not discharge the paper-to-local correspondence obligation.

For every component $u \in \{+, -, \mathrm{net}\}$, define its averaged cumulative by

$$I_\alpha^u(P) = \sum_{s,t:P(s,t)>0} P(s,t) i_\alpha^u(s,t;P).$$

For $u = +$, the absence of t from the local logarithm is visible, but this average initially uses joint weights.

Because

$$P(E_\alpha(s) \times \mathcal{T}) = \sum_{s' \in E_\alpha(s)} P_S(s'),$$

the finite sum can be regrouped over t:

$$\begin{aligned} I_\alpha^+(P) &= \sum_{s,t:P(s,t)>0} P(s,t) \left[-\log \sum_{s' \in E_\alpha(s)} P_S(s') \right] \\ &= \sum_{s:P_S(s)>0} P_S(s) \left[-\log \sum_{s' \in E_\alpha(s)} P_S(s') \right] \\ &=: G_\alpha(P_S). \end{aligned}$$

The load-bearing equality is $\sum_t P(s,t) = P_S(s)$. The theorem therefore proves a factorization

$$P_{S,T} \longmapsto P_S \longmapsto G_\alpha(P_S).$$

2.1 Exact invariance consequence

Standing assumptions for this consequence. Let P and Q have the same finite source alphabet and the same complete joint source marginal. Their finite target alphabets may differ, because the target has already been marginalized out. Then

$$P_S = Q_S \quad \Longrightarrow \quad I_\alpha^+(P) = I_\alpha^+(Q).$$

Define the complete informative cumulative vector by $C^+(P) := (I_{\alpha_i}^+(P))_{i=0}^{17}$. For one literally fixed finite matrix M, linearity gives

$$\Pi^+(P) = MC^+(P) = MC^+(Q) = \Pi^+(Q).$$

Calling M a Möbius inverse requires a separate proof that it is the inverse for the intended carrier, order, orientation, and key registry.

2.2 What was fixed and what was allowed to change

Fixed: source alphabets, source-only event family, complete source marginal, logarithm base, units, and any transformed-coordinate matrix. Allowed to change: target alphabet and the allocation of each source tuple's joint mass among target values—equivalently, a target kernel on positive-mass source tuples. Not allowed: changing the source event, quantizer, source alphabet, lattice, coordinate order, or matrix while calling the result the same theorem.

2.3 Probability semantics and the algebraic theorem

The executable routes use integer counts and exact positive rational products. The law-level proof above instead concerns exact real probabilities and logarithms. Their correspondence requires the count-to-empirical-law bridge in Section 3; neither representation may silently stand in for the other.

The generic Lean theorem at the Lean informative-invariance development formalizes supplied finite events and a supplied linear transform. It does not by itself prove that those supplied objects equal the publication objects.

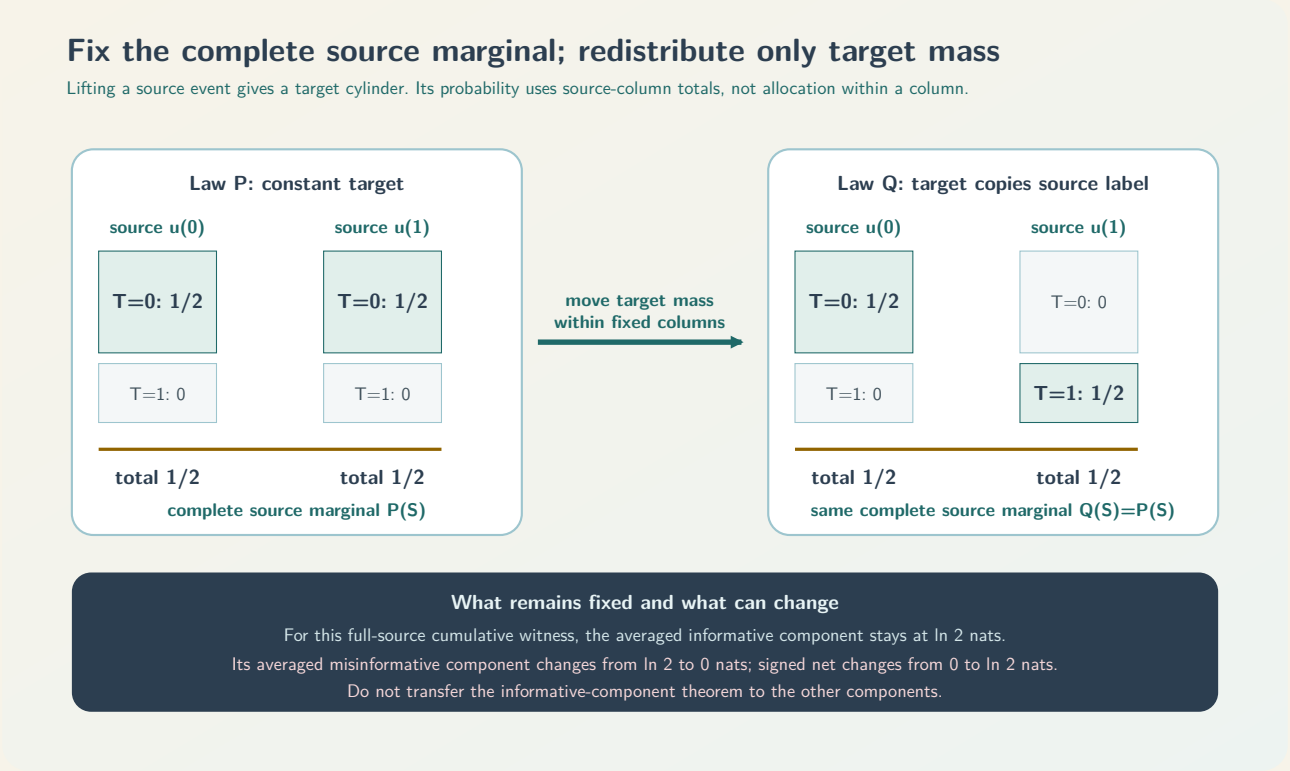
2.4 Retained prohibited-transfer witness

Standing assumptions for this counterexample. Take source support $u^{(0)} = (0,0,0)$ and $u^{(1)} = (1,1,1)$, each with probability $1/2$, and use the full-source cumulative event $\alpha = \{\{1,2,3\}\}$. Compare one constant target with a target that copies the source-state label. Then

$$I^{+}_{\{\{1,2,3\}\}} = H(S) = \log 2$$

under both laws, while

$$I^{-}_{\{\{1,2,3\}\}} : \log 2 \longrightarrow 0, \qquad I^{\mathrm{net}}_{\{\{1,2,3\}\}} : 0 \longrightarrow \log 2.$$



This is not a missing proof for minus/net invariance. It is a counterexample to that invariance.

Standing assumptions for the separate-marginals counterexample. On the common binary source alphabet, let

$$P_S = \tfrac{1}{2}\delta_{000} + \tfrac{1}{2}\delta_{111},$$

and

$$Q_S = \tfrac{1}{4}(\delta_{000} + \delta_{011} + \delta_{101} + \delta_{110}).$$

Every individual source is Bernoulli(1/2) under both laws, but at the full-source event $\alpha = \{\{1,2,3\}\}$,

$$I^+_{\{\{1,2,3\}\}}(P) = \log 2, \qquad I^+_{\{\{1,2,3\}\}}(Q) = \log 4.$$

Thus the theorem factors through P_{S_1,S_2,S_3}, not through the list of one-source marginals; those separate marginals do not determine the complete informative vector.

2.5 What remains after the factorization is proved

The short algebra does not close the following obligations:

- independent correspondence between local source events and the paper-defined events;
- concrete proof-assistant completeness and order of the intended 18-position carrier;
- a second non-importing formal construction of the intended zeta orientation and Möbius inverse;
- bytes-to-counts and counts-to-law decoding;
- compiled Rust or binary64 refinement;
- sampling assumptions, estimator calibration, causality, and application validity.

The defensible description is an elementary but reusable law-level lemma with explicit scope, a mechanized generic algebraic layer, and a retained falsifier for a tempting false transfer.

2.6 Relation to continuity

Standing assumptions for this subsection. Let P and Q be probability laws on finite source-target products with one declared common finite source alphabet, the same supplied source-only event family, and natural logarithms. For every $s \in \mathcal{S}$, the corresponding supplied event is anchored: $s \in E_\alpha(s)$. Their target alphabets may differ. Let P_S and Q_S be their complete joint source marginals, and let

$$\eta_S = d_{\mathrm{TV}}(P_S, Q_S) = \frac{1}{2}\sum_s |P_S(s) - Q_S(s)|.$$

Define overlap $r^\cap_s = \min\{P_S(s), Q_S(s)\}$, residuals $a_s = P_S(s) - r^\cap_s$ and $b_s = Q_S(s) - r^\cap_s$, and

$$E^S_\vee = \max\left\{ -\sum_{a_s>0} a_s \log a_s, -\sum_{b_s>0} b_s \log b_s \right\}.$$

These definitions imply $0 \le \eta_S \le 1$, $a_s, b_s \ge 0$, $\sum_s a_s = \sum_s b_s = \eta_S$, and $a_s b_s = 0$ for every s. Thus $E^S_\vee$ is the larger of two subprobability-residual entropy sums, with the zero terms omitted; it is not the Shannon entropy of either normalized residual law.

For $J \ge 1$ and $0 < \eta < 1$, define

$$g_J(\eta) = (1-\eta)\log\left(1 + \frac{J\eta}{1-\eta}\right),$$

with $g_J(0) = g_J(1) = 0$. The separately proved support-change theorem then gives, for $J_\alpha = |\alpha|$,

$$|I^+_\alpha(P) - I^+_\alpha(Q)| \le E^S_\vee + g_{J_\alpha}(\eta_S).$$

For one fixed matrix M,

$$|\Pi^+_i(P) - \Pi^+_i(Q)| \le \sum_\alpha |M_{i\alpha}| \left[E^S_\vee + g_{J_\alpha}(\eta_S)\right].$$

The support-change theorem applies because every antichain branch is the coordinate-projection equivalence class

$$B_a(s) = \{s' : \forall i \in a,\ s'_i = s_i\},$$

the supplied Sx event is exactly $E_\alpha(s) = \bigcup_{a\in\alpha} B_a(s)$, and therefore the number of branches in that theorem is $J_\alpha = |\alpha|$. No different event geometry is being smuggled into the continuity bound.

There is a further, precisely scoped **total-variation radius sharpening** when a joint-law radius is also available. For this comparison only, require P and Q to be laws on the *same* finite source-target alphabet $\mathcal{S} \times \mathcal{T}$, with the same labeling of both coordinates and the same supplied anchored event family. Define

$$\eta_{ST} = d_{\mathrm{TV}}(P, Q) = \frac{1}{2}\sum_{s,t} |P(s,t) - Q(s,t)|.$$

Marginalization is a contraction in total variation:

$$\begin{aligned} 2\eta_S &= \sum_s \left|\sum_t (P(s,t) - Q(s,t))\right| \\ &\le \sum_{s,t} |P(s,t) - Q(s,t)| = 2\eta_{ST}. \end{aligned}$$

Thus the factorization permits the informative-coordinate analysis to use the complete-source radius η_S instead of the joint source-target radius η_{ST}. This is a sharpening of the input radius in the non-strict sense $\eta_S \le \eta_{ST}$; equality can occur, for example when the target alphabet is a singleton, so no strict improvement is guaranteed. It is not a theorem that the complete displayed right-hand side is always numerically smaller than every joint-law bound: the residual-entropy term changes under marginalization, and g_J is not globally monotone. Finally, this factorization-based sharpening applies only to the informative cumulatives and one literally fixed linear transform of them. It does not transfer to the misinformative or signed-net components, which retain target-conditioned dependence as the counterexample in Section 2.4 shows.

The function g_J is not globally monotone, so a radius bound must not be substituted blindly. This is a deterministic stability statement, not a confidence bound. A separate statistical theorem is required to justify a random radius for an empirical law. The full theorem and its negative results are in `SUPPORT_CHANGE_TOLERANT_AVERAGED_SXPID_CONTINUITY.md`.

3. Exact finite-count formulation used by the bounded audit

Standing assumptions for this section. There are exactly three ordered binary sources and one binary target. A labelled 16-cell table has nonnegative integer counts c_z and total $N = \sum_z c_z > 0$. Let $\mathcal{Z}_+ = \{z : c_z > 0\}$. Every supported state occurs once in the keyed representation and is weighted by its count, not uniformly over distinct keys.

For supported $z = (s, t)$, first define the lifted events

$$E_\alpha(z) = \{(s', t') : s' \in E_\alpha(s),\ t' \in \{0, 1\}\}, \qquad T(z) = \{(s', t') : t' = t\}.$$

Thus $E_\alpha(z)$ is the lifted source cylinder and $T(z)$ is the target fibre. The count vector induces the exact empirical law

$$\hat{P}_c(z') = \frac{c_{z'}}{N}.$$

Define the exact integer masses

$$U_{\alpha,z} = \sum_{z' \in E_\alpha(z)} c_{z'}, \quad V_{\alpha,z} = \sum_{z' \in E_\alpha(z) \cap T(z)} c_{z'}, \quad T_z = \sum_{z' \in T(z)} c_{z'}.$$

Because the anchored state z belongs to all required events,

$$0 < c_z \le V_{\alpha,z} \le U_{\alpha,z} \le N, \qquad V_{\alpha,z} \le T_z \le N.$$

The inequalities are not inferred from the bounded enumeration. At the keyed row, every equality inside every selected branch is true and the target equals itself, so that row contributes the positive amount c_z to V, U, T_z, and N. For every comparison row,

$$\mathbf{1}\{E_\alpha \cap T(z)\} \le \mathbf{1}\{E_\alpha\}, \qquad \mathbf{1}\{E_\alpha \cap T(z)\} \le \mathbf{1}\{T(z)\}.$$

Multiplying by the nonnegative row count and summing proves the remaining inequalities. This indicator argument works for arbitrary finite categorical alphabets because each comparison row is reduced to three source-equality bits and one target-equality bit. It assumes a valid finite count table; it is not a unique bytes-to-table decoder proof.

Direct summation under $\hat{P}_c$ gives the promised count-to-law bridge:

$$\hat{P}_c(E_\alpha(z)) = \frac{U_{\alpha,z}}{N}, \qquad \hat{P}_c(E_\alpha(z) \cap T(z)) = \frac{V_{\alpha,z}}{N}, \qquad \hat{P}_c(T(z)) = \frac{T_z}{N}.$$

Substitution into the three law-level definitions in Section 2 gives the well-defined local count forms

$$i_\alpha^+(z) = \log \frac{N}{U_{\alpha,z}},$$

$$i_\alpha^-(z) = \log \frac{T_z}{V_{\alpha,z}},$$

and

$$i_\alpha^{\mathrm{net}}(z) = \log \frac{NV_{\alpha,z}}{U_{\alpha,z}T_z}.$$

For each cumulative, the exact positive-rational products are

$$Q_\alpha^+ = \prod_{z\in\mathcal{Z}_+} \left(\frac{N}{U_{\alpha,z}}\right)^{c_z},$$

$$Q_\alpha^- = \prod_{z\in\mathcal{Z}_+} \left(\frac{T_z}{V_{\alpha,z}}\right)^{c_z},$$

and

$$Q_\alpha^{\mathrm{net}} = \frac{Q_\alpha^+}{Q_\alpha^-}.$$

Under these assumptions the averaged cumulative is

$$\begin{aligned} C_\alpha^u &= \frac{1}{N}\sum_{z\in\mathcal{Z}_+} c_z i_\alpha^u(z) \\ &= \frac{1}{N}\log Q_\alpha^u. \end{aligned}$$

Standing assumptions for the transformed products. Use the fixed integer Möbius inverse M from Section 1.3 and the positive cumulative products above. For atom key γ, define

$$Q_{\gamma,\mathrm{atom}}^u = \prod_\alpha (Q_\alpha^u)^{M_{\gamma\alpha}}, \qquad \Pi_\gamma^u = \frac{1}{N}\log Q_{\gamma,\mathrm{atom}}^u.$$

Negative entries of M create reciprocal factors but no zero or undefined product. Since every product is strictly positive and $N > 0$, exact classification uses

$$Q = 1 \iff \frac{1}{N}\log Q = 0,$$

$$Q > 1 \iff \frac{1}{N}\log Q > 0, \qquad Q < 1 \iff \frac{1}{N}\log Q < 0.$$

No floating-point tolerance participates in this sign/zero decision.

4. Why there are 20,348 tables

Standing assumptions for this count. Treat all 16 binary joint cells as labelled and count every nonnegative integer vector with total mass between one and five. For one fixed N, write the N observations as stars and use 15 separators to split them among the 16 labelled cells. Choosing the star/separator arrangement gives $\binom{N+15}{15}$ vectors. Pascal's identity in telescoping form, $\binom{k}{15} = \binom{k+1}{16} - \binom{k}{16}$, then gives

$$\sum_{N=1}^{5} \binom{N+15}{15} = \binom{21}{16} - 1 = 20{,}348.$$

The totals contribute respectively

$$16,\ 136,\ 816,\ 3{,}876,\ 15{,}504.$$

Only 20,164 vectors represent primitive rational laws; 184 are integer rescalings. The latter count is also explicit: totals two, three, and five each have 16 nonprimitive point-mass vectors, while the nonprimitive total-four vectors are exactly the 136 vectors obtained by doubling every total-two vector. Hence $16 + 16 + 136 + 16 = 184$. The audit nevertheless retains all 20,348 vectors because replicated counts are executable regression cases. There are no full-support 16-cell laws in this sparse domain because $N \leq 5$.

With 108 expressions per table, each route evaluates

$$20{,}348 \times 108 = 2{,}197{,}584$$

strictly positive exact products.

5. The two exact routes and what “agreement” means

Standing assumptions for this evidence statement. The receipt is accepted only after its schema, source bindings, command statuses, stdout/stderr digests, normal/optimized parity, and source-state checks pass. Under that recorded boundary, the primary route and the independently implemented exact route emitted the same route-neutral v2 expression-stream SHA-256:

```
20c234cc664ad903aa66689d33d95b2db5bca5da3b0f9ee0b497d1246e3139b8
```

The primary and independent routes also emitted identical six-block exact sign/zero censuses. They are implementation-disjoint under shared semantics, not logically independent: both retain the human paper transcription, declared registry, Python runtime class, and other documented premises.

The routes differ in their executable mechanisms:

Layer	Primary route	Independent route
Table enumeration	Recursive weak compositions	Lexicographic stars-and-bars unranking
Exact arithmetic	Python `Fraction`	Signed prime-exponent vectors
Event mass	Direct membership scan	Cylinder marginals and inclusion--exclusion
Atom transform	Fixed exact inverse-product rows	Recursive subtraction on the declared poset
Native stream	Primary-specific framing	Independent-route-specific framing

Because $1 \le N \le 5$, every positive integer mass entering a ratio belongs to $\{1, 2, 3, 4, 5\}$. Its prime factorization therefore uses only 2, 3, and 5, making the second route's exponent vector exact on this bounded domain. This representation argument would have to change for a larger total-count bound.

Both routes separately serialize each freshly computed result into the same neutral-v2 frame. A frame binds the table ordinal, all 16 labelled counts, expression index, ASCII key, exact exponents of 2, 3, and 5, and the sign class. Neither route reads a per-table answer key or imports a pid-rs floating-point result. Expected aggregate pins are compared only after the route has recomputed and streamed every table-expression pair.

The receipt does not claim a direct record-by-record comparison of two retained output streams. It records agreement through one neutral stream digest plus the six census blocks. SHA-256 binds bytes under the stated host assumptions; it is not authenticity, authorship, or scientific truth.

A third lane lexically examines the Rust sources. It checks declared source anchors and configuration boundaries but computes zero Rust numerical values. It therefore does not establish Rust parsing, name resolution, compilation, execution, binary64 refinement, or keyed numerical agreement.

Source inspection and hostile tests found no per-table answer table, endpoint-specific forced-value branch, or cross-route import of numerical output. Those controls reduce these risks; they cannot eliminate fabrication, a shared mistranscription, a coordinated erroneous change followed by resealing the expected aggregate hashes, a framing defect, or compromise of the common runtime/host. The correct description is therefore "implementation-disjoint under shared semantics," not "independent proof."

6. Exact sign/zero census

Standing assumptions for this table. Within each block, every pair consisting of one labelled count vector and one of the 18 antichain keys has unit weight. Thus each table contributes 18 classifications to each block, for 366,264 classifications per block. These classifications are not weighted by prevalence or probability. Every classification is made by exact comparison of a strictly positive rational product Q with one.

Stage and component	Negative	Positive	Zero	Total
cumulative informative	0	321,856	44,408	366,264
cumulative misinformative	0	278,984	87,280	366,264
cumulative net	29,496	252,816	83,952	366,264
atom informative	0	145,100	221,164	366,264
atom misinformative	0	71,468	294,796	366,264
atom net	31,284	96,768	238,212	366,264

Every row totals $20{,}348 \times 18 = 366{,}264$. Negative signed-net cumulatives and atoms are exposed, not clamped. Informative and misinformative component entries are nonnegative on this bounded corpus; this census is not itself the general component-nonnegativity theorem.

Standing assumptions for one retained negative witness. Give count one to exactly

$$(S_1, S_2, S_3, T) \in \{(1, 0, 1, 1), (1, 1, 0, 1), (1, 1, 1, 0)\},$$

and count zero to every other labelled cell, so $N = 3$. At atom key `02+04`, the exact informative, misinformative, and signed-net products are respectively

$$\frac{9}{4}, \qquad 4, \qquad \frac{9}{16}.$$

Hence

$$\Pi^{\mathrm{net}}_{02+04} = \frac{1}{3}\log\frac{9}{16} < 0.$$

This is a finite categorical functional value, not a causal or population interpretation.

6.1 The 108 expressions are algebraically dependent

The receipt records 18 cumulative and 18 atom net-equals-plus-minus identities and 54 zeta reconstruction identities. It explicitly does not adjudicate a "base rank" or statistical independence of components. The expanded registry is an audit design, not a claim of 108 mathematical degrees of freedom.

7. Formal, executable, and receipt evidence

7.1 Factorization-result evidence

The finite-event and fixed-transform algebra is checked in Lean, with the paper-to-supplied-event correspondence retained as an external premise. The parity checker compares Lean declarations with the executable theorem inventory. This supports the algebraic implication within its theorem statement; it does not validate the publication transcription or estimator semantics.

7.2 Bounded-audit evidence

The authoritative receipt is the source-bound bounded-audit receipt. It binds:

- the receipt schema and exact source inputs;
- primary and independent exact lanes in normal and optimized Python;
- both mutation/self-test suites in normal and optimized Python;
- the lexical Rust route and its self-test;
- stdout, stderr, source, runtime, Git, and host observations;
- the exact domain, digests, census, algebraic dependencies, and nonclaims.

The receipt supplies bounded local execution provenance. It explicitly supplies no external custody, replay, trusted timestamp, authenticity, general theorem, or semantic-transfer authority.

7.3 Revision-5 semantic-bridge evidence

The full source correspondence and its machine record bind the owner-controlled paper reading and executable reconstruction. Starting from three source bits, the checker derives:

Reconstruction	Result
Nonempty antichains	18
Ordered carrier pairs	324
True order/zeta entries	129
Nonzero integer Möbius entries	65
Source-label automorphisms	6
Source-event cases	144
Source-event/target cases	288

Both exact inverse products are required. Every source permutation must preserve the carrier, order, and event truth after the equality coordinates are relabeled with the sources. The event truth cases distinguish OR across collections, AND within a collection, and target intersection.

The executable checks all Boolean premises used by the generic count proof. The written proof, not a finite enumeration of every possible count vector, supplies the step that multiplies those indicator

inequalities by arbitrary nonnegative row weights and sums. The checker therefore must not be described as exhaustively running all finite alphabets or all count vectors.

A separate exact compatibility edge binds the frozen conventions and both older bounded-route files by SHA-256 and byte count. Without importing either route's computation, the new reconstruction must equal the conventions and primary-route registries at all 18 carrier keys, all 324 zeta entries, and all 65 nonzero Möbius coefficients; it must also equal the second route's key registry and 129/65 lattice censuses. This detects silent registry drift. It is not logical independence, event-implementation comparison, bounded-audit execution, or Rust refinement.

The hostile self-test explicitly covers all seven failure identifiers preserved in the inert historical-checker archive. That archive retains seven mutation recipes and fourteen mode-specific observations from two superseded checkers; it is not executed by the current gate and earns zero closure credit. Under normal and optimized Python, the current self-test executes two baselines, four alternate-input rejections, 28 coherently resealed record rejections, 12 semantic-source rejections, six frozen compatibility-file drift rejections, four coherently resealed compatibility-literal rejections, and two document-drift rejections. Four of the 28 record executions cover two exact JSON type substitutions in both modes: Boolean `false` for integer `0`, and floating `5.0` for integer `5`. The four separately counted compatibility-reseal executions cover Boolean `False` for an integer Möbius-tuple entry and floating `129.0` for integer census `129`, each in both modes. The pre-correction committed v4 checker accepted all four substitutions after coherent owner-controlled reseals because ordinary Python equality compares those type-distinct values as equal. The current checker compares all verdict-bearing record fields and all six parsed compatibility registries recursively by exact type, shape, and value. The exact pre-correction bytes, minimal reproductions, repair, and residual limits remain in the retained type-coercion failure record. This was a verification-chain defect, not a change to the event semantics, finite reconstruction, or open Program status.

It also requires two *accepted* coordinated-reseal negative controls. In an isolated copy, the test changes correct prose, updates its machine binding, and reseals both checker digest literals. The checker passes. This is an important limitation, not a hidden false green: byte-binding and local reconstruction cannot interpret natural-language mathematics or stop their owner from rewriting all owner-controlled evidence together. The accepted mutation receives no correctness, source, review, authenticity, or custody credit.

8. Current certificate-obligation status

The current adjudication is decision record 3, selected by the evidence-adjudication index. The complete certificate program remains open. In particular:

Obligation	Current status
Paper-to-local event correspondence	Owner-controlled revision-5 source map and fresh acquisition replay recorded; independent acquisition/review open
Exact source-marginal algebra	Analytically and generically formally supported
Bounded two-route exact agreement	Supported on the declared $N \leq 5$ corpus
Concrete three-source carrier/order/event executable reconstruction	Supported in one owner-controlled Python route
Concrete three-source carrier/order proof in two formal systems	Open; the executable bridge is not a substitute
Exact logarithm magnitude enclosure	Open for this 108-expression package
Keyed compiled-Rust numerical refinement	Open
Two-route executable product audit beyond binary alphabets or $N \leq 5$	Open
Population/statistical validity	Outside this deterministic claim

A complete certificate would need canonical decoding, exact event/count reconstruction, the concrete carrier and order, exact products, directed logarithm intervals, resource preflight, keyed Rust refinement, mutation evidence, and independently replayed custody. The bounded receipt is evidence for some of those obligations, not a substitute for all of them.

9. Computational cost and estimator impact

The factorization is an algebraic reuse result. Once the complete source marginal and source-event probabilities are available, changing only the target allocation requires no recomputation of the informative cumulatives. A fixed transformed informative vector is one matrix-vector operation.

The bounded audit is intentionally expensive but finite: each exact route evaluates 2,197,584 products plus registry, mutation, and framing controls. Big-integer cost depends on count size and intermediate bit length; calling the number of scalar expressions finite is not a constant-bit-cost claim.

No new estimator is required to evaluate these deterministic identities on a *supplied* finite categorical probability law or count table. Inferring a population categorical Sx quantity from sampled data still requires an explicitly specified estimator: the empirical plug-in law is one choice, and its bias, consistency regime, uncertainty, and dependence assumptions remain separate statistical questions. Moving to continuous variables requires a matching continuous estimand and estimation theory. Quantization changes the estimand and must be fitted and reported rather than treated as a transparent bridge.

Practical implementations should cache source-only event counts, use exact keys rather than array positions, preflight memory and projected integer size, and reserve exact products for validation or escalation when a fast floating path is insufficient. Any optimization of the exact audit path must retain the same event semantics and the exact positive-rational product computed from event counts and compared with one—not an answer table, tolerance rule, or imported Rust output. A fast production path need not compute that product on every call, but it must preserve the declared semantics and remain checked against exact fixtures in the domains for which agreement is claimed.

10. Why these results are useful

10.1 Mathematical use

- The factorization isolates precisely which part of categorical Sx depends only on the complete source marginal.
- The target-allocation counterexample prevents a false extension to misinformative and net terms.
- The 18/108/166 crosswalk prevents a bookkeeping vector from being mistaken for a lattice.
- The Equation (4)/Equation (6) witness prevents individual-source OR from replacing conjunction inside one joint-source collection.
- The definition-generated carrier, order, and two-sided inverse reduce reliance on one frozen hand-written matrix.
- Exact products make zero and sign questions algebraic rather than tolerance-dependent.
- The census demonstrates that negative signed-net atoms occur inside the declared finite domain.

10.2 Engineering use

- Cache informative coordinates across target permutations that preserve source rows.
- Test event extraction, component signs, Möbius orientation, and net subtraction separately.
- Compare implementations by stable antichain key, not positional array index.
- Require semantic connective and target-intersection mutants before trusting two arithmetic implementations that share a transcription.
- Use the finite corpus as a strong regression gate for a declared transcription.
- Preserve negative and exact-zero cases as mandatory mutation targets.

10.3 Example application interpretations

1. **Neuroscience target surrogates.** When spike-pattern sources are fixed and behavioural labels are permuted, the categorical informative coordinates can be reused. Exchangeability and the validity of the surrogate design remain separate statistical obligations.
2. **Distributed-system monitoring.** If source-state frequencies are unchanged while an outcome label allocation changes, the factorization distinguishes reusable source geometry from target-conditioned computation. It does not make the PID causal.
3. **Genomic categorical screens.** Exact-count fixtures can detect lattice-key or sign regressions in a fast implementation. Population inference still needs a sampling model and multiplicity control.
4. **Cross-library comparison.** The 108-key registry can diagnose where two Sx transcriptions disagree. It cannot equate Sx with BROJA, $I_{\min}$, or another PID.

11. Explicit nonclaims and negative results

- The factorization does not extend in general to misinformative or signed-net components.
- Three individual source marginals do not determine the complete informative vector.
- The bounded audit is not an arbitrary-alphabet or arbitrary-total theorem.
- Bounded arithmetic-route agreement alone is not paper-to-code correspondence and not logical independence.
- Owner-controlled source correspondence is not independent review, authenticity, or a machine interpretation of the paper.
- The accepted coordinated-reseal negative control shows that local byte bindings cannot certify natural-language meaning.
- A disposable-checkout integrity failure occurred during an exact PDF-check attempt. The incident record preserves the observations, bounded recovery, unresolved causal status, defensive hardening, and a clean canary result; it neither changes the mathematics nor proves that an in-scope script caused the damage.
- The lexical Rust lane is not compiled numerical refinement.
- Exact products classify sign and zero but do not enclose nonzero logarithm magnitude.
- No binary64 correctness, portable logarithm, estimator calibration, confidence coverage, population PID, causal interpretation, or application decision is established.
- Negative signed-net atoms are legitimate and must not be clamped.
- Within each census block, every labelled-table/antichain-key pair has unit weight. Each table therefore contributes 18 classifications; the census is not a probability distribution over empirical laws or datasets.
- The receipt provides repository custody, not authenticity, authorship, priority, or attestation.

12. Reproduction entry points

Run the authoritative receipt checker and its mutation suite with the required isolated Python flags:

```
python3 -I -S -B scripts/check-sxpid3-bounded-keyed-scalar-audit-expressions-receipt-v1.py
 ↪  --committed-or-preserved
python3 -O -I -S -B
 ↪  scripts/check-sxpid3-bounded-keyed-scalar-audit-expressions-receipt-v1.py
 ↪  --committed-or-preserved
```

```
python3 -I -S -B
 ↪  scripts/check-sxpid3-bounded-keyed-scalar-audit-expressions-receipt-v1-self-test.py
python3 -O -I -S -B
 ↪  scripts/check-sxpid3-bounded-keyed-scalar-audit-expressions-receipt-v1-self-test.py
```

The committed-or-preserved mode intentionally requires complete Git history and an exactly clean worktree. Run it from a clean checkout of the commit under review; a shallow or dirty checkout is a fail-closed result, not evidence failure to be bypassed.

The source-marginal algebra and its formal/hostile checks use the separate pinned Lean lane:

```
(cd audit/formal/lean && lake exe cache get)
python3 -I -S -B scripts/check-lean-sxpid3-informative-invariance.py
python3 -O -I -S -B scripts/check-lean-sxpid3-informative-invariance.py
python3 -I -S -B scripts/check-lean-sxpid3-informative-invariance-self-test.py
python3 -O -I -S -B scripts/check-lean-sxpid3-informative-invariance-self-test.py
python3 -I -S -B scripts/check-lean-sxpid3-informative-invariance-parity.py
```

The parity checker binds exact source and dependency bytes and compares the declared theorem inventory with kernel-checked output. It does not close the publication-object correspondence or the concrete Rust/count refinement.

The revision-5 source map and concrete finite semantic reconstruction use:

```
python3 -I -S -B scripts/check-sxpid3-mgw-v5-program-a-semantic-bridge-v4.py
python3 -I -S -B -O scripts/check-sxpid3-mgw-v5-program-a-semantic-bridge-v4.py
python3 -I -S -B scripts/check-sxpid3-mgw-v5-program-a-semantic-bridge-v4-self-test.py
python3 -I -S -B -O scripts/check-sxpid3-mgw-v5-program-a-semantic-bridge-v4-self-test.py
```

The production checker has no alternate-input option. The self-test copies canonical files into isolated temporary repository shapes, mutates those copies, and preserves the coordinated-reseal pass as a declared boundary. Passing these commands does not close independent source review, formal semantics, parsing, logarithm enclosure, or Rust refinement.

The underlying exact lanes and their hostile tests are:

```
python3 -I -S -B scripts/check-sxpid3-bounded-full-coordinates.py
python3 -O -I -S -B scripts/check-sxpid3-bounded-full-coordinates.py
python3 -I -S -B scripts/check-sxpid3-bounded-full-coordinates-self-test.py
python3 -O -I -S -B scripts/check-sxpid3-bounded-full-coordinates-self-test.py
python3 -I -S -B scripts/check-sxpid3-all108-independent.py
python3 -O -I -S -B scripts/check-sxpid3-all108-independent.py
python3 -I -S -B scripts/check-sxpid3-all108-independent-self-test.py
python3 -O -I -S -B scripts/check-sxpid3-all108-independent-self-test.py
python3 -I -S -B scripts/check-sxpid3-p5-rust-source-route.py
python3 -O -I -S -B scripts/check-sxpid3-p5-rust-source-route.py
python3 -I -S -B scripts/check-sxpid3-p5-rust-source-route-self-test.py
python3 -O -I -S -B scripts/check-sxpid3-p5-rust-source-route-self-test.py
```

The two-source count/event bridge is documented in the two-source bridge note. It is adjacent evidence, not a proof of the full three-source executable route.

References

- Abdullah Makkeh, Aaron J. Gutknecht, and Michael Wibral, "Introducing a Differentiable Measure of Pointwise Shared Information," *Physical Review E* 103, 032149 (2021), doi:10.1103/PhysRevE.103.032149. Equation anchors in this report were checked against arXiv:2002.03356v5.
- Aaron J. Gutknecht, Michael Wibral, and Abdullah Makkeh, "Bits and Pieces: Understanding Information Decomposition from Part-Whole Relationships and Formal Logic," *Proceedings of the Royal Society A* 477, 20210110 (2021), doi:10.1098/rspa.2021.0110.
- Paul L. Williams and Randall D. Beer, "Nonnegative Decomposition of Multivariate Information" (2010), arXiv:1004.2515. This is the source for the lattice carrier, not for the MGW redundancy value.
- Nils Bertschinger, Johannes Rauh, Eckehard Olbrich, Jürgen Jost, and Nihat Ay, "Quantifying Unique Information," *Entropy* 16, 2161–2183 (2014), doi:10.3390/e16042161. It identifies the distinct BROJA family and is not transferred into the categorical Sx result.
- David A. Ehrlich, Kyle Schick-Poland, Abdullah Makkeh, Felix Lanfermann, Patricia Wollstadt, and Michael Wibral, "Partial Information Decomposition for Continuous Variables Based on Shared Exclusions," *Physical Review E* 110, 014115 (2024), doi:10.1103/PhysRevE.110.014115. Its continuous estimator is not identified with this finite categorical audit.
- Gian-Carlo Rota, "On the Foundations of Combinatorial Theory I. Theory of Möbius Functions," *Zeitschrift für Wahrscheinlichkeitstheorie und Verwandte Gebiete* 2, 340–368 (1964), doi:10.1007/BF00531932.
- OEIS Foundation, "A000372: Dedekind numbers," oeis.org/A000372. Only the standard small antichain counts are used; the PID carrier restrictions are stated separately above.